MargDarshak-AI

High-Level Design

Components, data flow and the decisions behind them

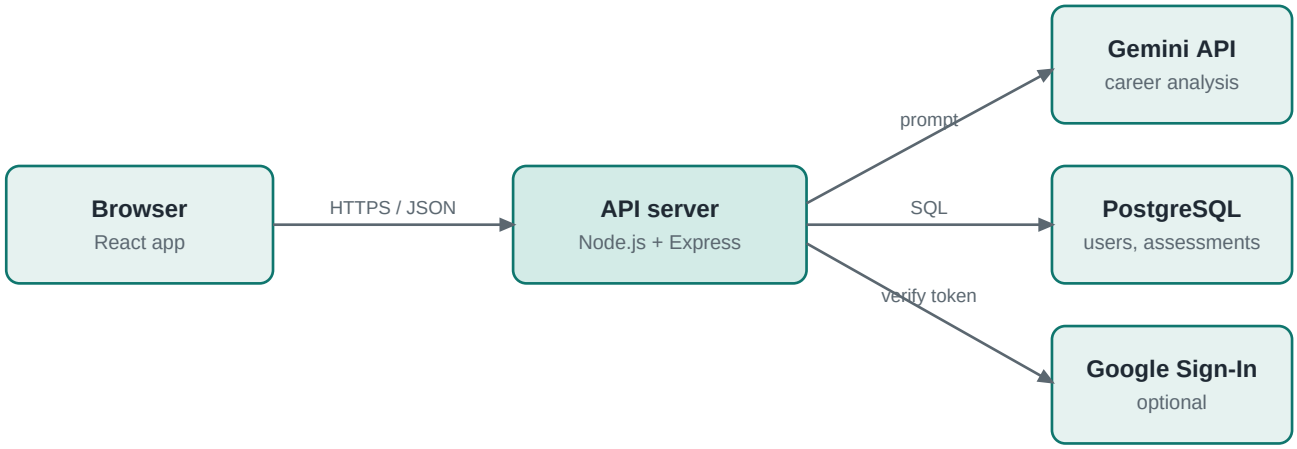
Status	Draft v1.0
Date	7 November 2026
Read with	PRD (what and why), LLD (how it is built)

1. Scope

This document describes the shape of MargDarshak-AI: the pieces, what each one is responsible for, how data moves between them, and the decisions behind that. It stays at the level of components. Tables, endpoints and code structure are in the LLD.

The system takes a student's assessment, asks a Gemini model for a structured career analysis, stores the result against the student's account, and lets them track progress on the roadmap it contains.

2. Architecture at a glance



3. Components

Component	Responsibility	Does not do
React app	Assessment form, dashboard, roadmap and progress views, sign-in screens. Renders whatever analysis the API returns.	Talk to Gemini or the database directly; hold secrets
Express API	Validates input, authenticates users, builds prompts, calls Gemini, validates the reply, reads and writes the database.	Render UI; trust anything from the client
Gemini	Turns the assessment into a structured analysis (careers, gaps, steps, roadmap, projects).	Store anything; see user identity
PostgreSQL	Accounts, saved assessments and analyses, progress, feedback.	Business logic beyond constraints and indexes

Component	Responsibility	Does not do
Google Sign-In	Proves a user's identity when enabled. The API verifies the token it issues.	Required for the app to work

4. Main flow: generating a result

1. The student fills the form in the browser and submits it with the number of careers wanted (5 to 10).
2. The API checks the session, validates the fields and trims anything oversized.
3. It builds one prompt from the answers, the optional Indian education context and the personal instructions.
4. It calls Gemini asking for JSON in a fixed shape.
5. It parses and checks the reply: right number of careers, required fields present, sensible lengths. One retry if the check fails.
6. The assessment and analysis are saved under the user's id, and the analysis is returned to the browser.
7. The dashboard draws itself from that JSON. Progress ticks later go back to the API one at a time.

5. Data, in broad strokes

Five things are stored. Each row belongs to exactly one user, and every query filters on that user.

Data	Notes
User	Email, name, password hash or Google identifier, created date
Assessment	The student's answers and instructions, requested career count, timestamp
Analysis	The model's JSON reply, stored as a document so the dashboard can be rebuilt exactly
Progress	One row per roadmap item per user: done or not, with timestamp
Feedback	Rating and comment tied to an analysis

6. Sign-in and security

- Email and password accounts. Passwords are hashed with bcrypt and never logged.
- Google sign-in appears only when the server has Google credentials configured; otherwise the button is hidden.
- Sessions use a signed token in an HTTP-only cookie, so page scripts cannot read it.
- Sign-out clears the cookie; switching accounts starts a clean session and cached data is dropped in the browser.
- All database access is parameterised. Every read and write is scoped by user id.
- API keys (Gemini, Google, database URL, token secret) live in server environment variables only.
- Rate limiting on sign-in and on the analysis endpoint, since the second one costs money.

7. Working with the AI model

The model is treated as an unreliable but useful collaborator. The design assumes it will sometimes be slow, wrong or badly formatted.

- **Structured output.** The prompt asks for JSON only, and the server checks the result against a schema before saving.
- **One retry.** If the reply fails validation, the API retries once with a short corrective note. After that it returns a friendly error.
- **Timeouts.** The call is cut off after a fixed limit so the user is never left on an endless spinner.
- **No personal identifiers in prompts.** The model gets the answers, not the name or email.
- **Honest framing.** The dashboard always shows the "starting point, not a guarantee" notice.

8. Deployment and configuration

- Frontend is built as static files and served from a CDN or static host.
- API runs as a single Node process behind HTTPS; it can scale out because sessions are stateless.
- PostgreSQL is a managed instance with daily backups.
- Separate environments for development and production with different keys.
- Configuration is by environment variable: database URL, token secret, Gemini key, optional Google client id, allowed origin.

9. Non-functional targets

Area	Target
Responsiveness	Pages load fast on a mid-range phone over 4G; analysis shows a progress state and finishes within the timeout
Reliability	A failed model call never loses the student's answers; they can retry without retyping
Privacy	Minimum data collected; user can delete their account and data
Cost	Analysis calls are rate limited per user; saved results are reused, not regenerated
Accessibility	Readable contrast, keyboard-friendly forms, works on small screens

10. Decisions and trade-offs

Decision	Why	What we give up
Store the analysis as JSON, not split into many tables	Dashboard can be rebuilt exactly; the model's shape can change without migrations	Harder to query across careers
One Gemini call per assessment	Simple, cheap, easy to reason about	Long output, so slower than small calls
Stateless cookie sessions	Easy to scale, no session store	Revoking a token before expiry needs extra work
Keep the AI call on the server	Key stays secret, prompts can be tuned without a release	Server carries the cost and the latency
Google sign-in optional	App works in any environment, even with no Google setup	Two sign-in paths to test

11. Known risks

- Model quality is the main product risk. Mitigation is validation, feedback and regular review of real outputs.
- A single API server is a single point of failure until a second instance is added.
- Young users raise the bar on privacy. Age policy and data deletion need a decision before launch.